Pantallas principales (vecino)

1. Login / selección de comunidad

- Campos:
  - Código comunidad (o enlace mágico).
  - Teléfono o email.
  - Nombre y piso/puerta la primera vez.
- Opciones técnicas: login por código enviado por SMS/email (sin contraseñas).

2. Home sencilla

- Secciones visibles:
  - Avisos (3 últimos).
  - Botón “Nueva incidencia”.
  - Botón “Reservar espacio”.
  - Botón flotante “Preguntar al secretario IA”.

3. Incidencias

- Lista de incidencias del vecino y, opcionalmente, de toda la comunidad (según ajustes de privacidad).
- Filtros: abiertas, en curso, resueltas.

Detalle de incidencia (vista):

- Campos:
  - Título.
  - Categoría (ascensor, garaje, limpieza, ruido, otros…).
  - Prioridad (baja/media/alta).
  - Estado (abierta, en curso, resuelta).
  - Descripción.
  - Fotos/vídeos adjuntos.
  - Fecha creación, últimas actualizaciones.
  - Comentarios (vecino ↔ admin).

Crear incidencia:

- Campos que rellena el vecino:
  - Título (texto corto).
  - Descripción.
  - Adjuntos.
- Cosas que hace la IA:
  - Clasifica en categoría y prioridad.
  - Genera mensaje estándar de confirmación.

4. Avisos / comunicados

- Lista de avisos ordenados por fecha: cortes de agua, juntas, limpieza garaje, etc.
- Cada aviso tiene: título, descripción, fecha de publicación y fecha de fin, adjuntos opcionales.

5. Reservas de espacios

- Seleccionar espacio: pista pádel, sala usos múltiples, barbacoa, plaza invitado, etc.
- Ver calendario sencillo (por días o slots horarios).
- Crear reserva: día, hora inicio/fin, notas.

Pantallas principales (admin/presidente)

6. Panel de incidencias

- Tabla con: comunidad, vecino, categoría, prioridad, estado, fecha.
- Acciones: cambiar estado, añadir nota interna, asignar a proveedor, cerrar.
- IA: sugerir respuestas rápidas, agrupar incidencias parecidas, generar informe mensual.

7. Gestión de avisos

- Crear aviso: escribe 2-3 frases tipo borrador.
- IA genera:
    - Versión formal para el portal.
    - Versión corta para WhatsApp/email.

8. Gestión de reservas

- Vista por espacio: quién reserva, cuándo, si hay conflictos.
- Posible configuración: bloques por día, duración máxima, límite de reservas por vecino.

9. Ajustes de comunidad

- Datos básicos: nombre comunidad, dirección, nº vecinos.
- Roles: administrador, presidente, vecinos.
- Reglas de uso: texto de normas que la IA usará como contexto para responder.

Modelo de datos mínimo (tablas / colecciones)

Puedes hacerlo en SQL o NoSQL; te lo pongo conceptual:

Tabla `communities`

- id
- name
- address
- code (para invitaciones)
- settings (JSON: límites reservas, si incidencias son públicas, etc.)

Tabla `users`

- id
- community_id
- name
- email / phone
- role (`neighbor`, `admin`, `president`)
- unit (piso/puerta)

Tabla `incidents`

- id
- community_id
- user_id (quien la abre)
- title
- description
- category (string)
- priority (string)
- status (`open`, `in_progress`, `closed`)
- created_at, updated_at

Tabla `incident_attachments`

- id
- incident_id
- file_url
- type (`image`, `video`)

Tabla `incident_comments`

- id
- incident_id
- user_id
- message
- created_at

Tabla `notices`

- id
- community_id
- title
- body
- short_body (versión IA para WhatsApp)
- start_date, end_date
- created_by

Tabla `spaces`

- id
- community_id
- name (pista pádel, sala juntas…)
- rules (texto / JSON)

Tabla `reservations`

- id
- space_id
- user_id
- community_id
- start_datetime
- end_datetime
- status (`confirmed`, `cancelled`)
- created_at

Tabla `faqs / docs`

- id
- community_id
- title
- body (normas, estatutos, info bancaria, etc.)

"Secretario IA": lógica básica

Contexto que le pasas al modelo IA en cada petición de chat o generación:

- Datos de la comunidad (nombre, dirección, IBAN, teléfonos de urgencias).
- Normas (tabla `faqs/docs`).
- Historial reducido de la conversación y, si aplica, de incidencias/avisos relevantes.

Casos de uso:

- Preguntas frecuentes:
    - Input: “¿cuándo se limpia el garaje?”
    - La IA busca en avisos recientes o documentos y responde sencillo.
- Redactar comunicados:
    - Input admin: “el martes 12 se limpia el garaje de 9 a 13, que no aparquen”
    - Output IA:
        - Comunicado formal.
        - Versión corta.
- Resúmenes de reuniones:
    - Input: texto largo de acta.
    - Output: puntos clave para vecinos (decisiones y fechas).

Flujos clave

Crear incidencia (vecino)

1. Vecino pulsa “Nueva incidencia”.
2. Rellena título + descripción + fotos.
3. Backend llama a IA para clasificar (categoría/prioridad) y generar texto de confirmación.
4. Guarda incidencia en BD y envía notificación al admin.

Crear aviso (admin)

1. Admin escribe borrador corto.
2. IA genera versión larga + corta.
3. Admin revisa y publica.
4. Vecinos reciben push/email/WhatsApp (según lo que soportes).

Reservar espacio

1. Vecino elige espacio y día.
2. Ve slots libres.
3. Selecciona hora; backend valida reglas (límite de reservas, conflictos).
4. Se guarda reserva y se envía recordatorio automático.

Cobro y planes

- Cada `community` tiene un plan (`basic`, `pro`) y fecha de renovación.
- Puedes empezar cobrando manual (transferencia) y luego integrar Stripe.
- Lógica simple: si comunidad no está pagada, bloquear nuevas incidencias/reservas pero dejar ver el histórico.

Arquitectura recomendada

- **Frontend:** Next.js 15 (React) con App Router, Tailwind para ir rápido.
- **Backend:**
  - API Routes / server actions de Next.js para lógica de negocio (incidencias, reservas, llamadas a IA).
  - Supabase como BaaS: Postgres, auth con magic link/email, storage para fotos, RLS para separar comunidades.
- **IA:**
  - Proveedor tipo OpenRouter / Gemini / lo que uses ya, a través de una API route "/api/assistant" que recibe prompt y contexto (normas, avisos, etc.).
- **Pagos:**
  - Stripe Billing con planes mensuales "Básico" y "Pro"; cada `community` está ligada a un `customer` y `subscription` de Stripe.

Esquema lógico

- Multi-tenant sencillo:
  - `organizations` = comunidades.
  - Usuarios pueden pertenecer a una comunidad (más adelante a varias).
- Autenticación: Supabase Auth con email (magic link) o usuario + contraseña; rol en tabla `profiles` (vecino, admin, presidente).
- Autorización: se filtra siempre por `community_id` y rol en cada consulta (RLS).

Roadmap por semanas (realista)

Semana 1 – Base técnica y auth

Objetivo: que un usuario pueda registrarse, entrar en una comunidad de prueba y ver una dashboard vacía.

- Configurar proyecto:
  - Crear app Next.js (`create-next-app`).
  - Conectar Supabase: URL y anon key, inicializar cliente en el proyecto.
- Auth básica:
  - Signup/login con email y magic link.
  - Tabla `profiles` enlazada a `auth.users` para guardar nombre, rol, piso/puerta.
- Modelo de datos mínimo en Supabase:
  - `communities`, `users/profiles`, `incidents`, `notices`, `spaces`, `reservations`.
- Páginas:
  - `/login`, `/app` (dashboard vacía), `/app/settings` sólo para admin/presidente.

Resultado: ya puedes invitar a un presidente a entrar, aunque aún no haya funciones.

Semana 2 – Incidencias + avisos (sin IA aún)

Objetivo: que una comunidad pueda registrar incidencias y publicar avisos.

- Incidencias:
  - Formulario "Nueva incidencia" (título, descripción, foto).
  - Listado de incidencias del usuario y, si eliges, de toda la comunidad.
  - Estados: abierta, en curso, resuelta; cambios desde panel admin.
- Avisos:
  - Panel admin para crear, editar y cerrar avisos.
  - Lista de avisos en home del vecino.
- Reglas de acceso:
  - Middleware de Next.js o layout protegido para `/app/**`.

Resultado: MVP útil sin IA, ya puedes enseñar algo y probar flujo con 1-2 comunidades.

Semana 3 – Reservas + "secretario IA" básico

Objetivo: añadir reservas de espacios y un primer chat/acción IA.

- Reservas:
  - Tabla `spaces` y `reservations`.
  - Pantalla para elegir espacio y ver calendario básico por día/slots.
  - Validaciones simples en backend (no solapar reservas; límite por usuario).
- IA – Modo 1 (FAQ y comunicados):
  - API route `/api/assistant` que recibe: pregunta del vecino + `community_id`.
  - El backend busca en `faqs/docs` y últimos `notices` de esa comunidad y se lo pasa al modelo en el prompt.
  - Devuelve respuesta y la pinta en un chat sencillo.
  - Botón en panel admin "Generar comunicado" donde escriben un borrador corto y la IA devuelve versión formal + corta.

Resultado: ya existe el "secretario IA" en la práctica, aunque básico, y reservas.

Semana 4 – Suscripciones y primeras ventas

Objetivo: poder cobrar 5-15 €/mes por comunidad y dejar listo para producción.

- Stripe Billing:
  - Crear productos/planes en Stripe: "Básico comunidades" y "Pro comunidades".

- API route `/api/stripe/create-checkout` que crea sesión de checkout para el admin o administrador de fincas.
- Webhook `/api/stripe/webhook` para actualizar en Supabase:
    - `stripe_customer_id`, `stripe_subscription_id`, `plan`, `status`.
- Lógica de acceso a features:
    - Si plan = Básico: sólo incidencias + avisos.
    - Si plan = Pro: activa reservas + IA.
    - Si `status` no pagado, mostrar aviso y limitar ciertas acciones.
- Preparar landing muy simple (puede ser otra app Next.js o una página pública en la misma):
    - Explicar beneficios, demo, precios y botón "Empezar gratis 14 días".

Resultado: SaaS listo para meter comunidades reales y facturar.

Opcional: acelerar con boilerplate

Si quieres ir a tope a funcionalidad y no perder tiempo en auth, billing y estructura, puedes:

- Usar un boilerplate Next.js + Supabase con billing ya montado (ej. MakerKit o similares), que ya trae: auth, multi-tenant, Stripe integrado y panel admin, para que tú sólo construyas módulos de incidencias/reservas.
- Aprovechar su estructura de organizaciones para que cada "organization" sea una comunidad y tú te centres en pantallas y lógica de negocio.

1. Estilo general

- Estilo: **SaaS moderno y limpio**, tipo Linear / Notion / Stripe dashboard, muy legible.
- Paleta:
    - Color primario: azul ligeramente violáceo (#3B82F6 aprox.).
    - Color secundario: verde suave para estados resueltos (#22C55E).
    - Fondo: gris muy claro (#F5F5F7) con tarjetas blancas.
    - Resaltados: uso discreto de glassmorphism solo en la barra lateral y el header.
- Tipografía:
    - Sans-serif moderna (similar a Inter / SF Pro / Roboto).
    - Títulos con peso medio/semibold, texto normal muy legible.
- Componentes:
    - Botones primarios claros, con esquinas ligeramente redondeadas.
    - Tarjetas para incidencias y avisos.
    - Badges para estados (Abierta, En curso, Resuelta) con colores diferentes.
    - Iconografía mínima y clara (no recargar).

2. Estructura de navegación

Diseña la app como **dashboard web** con:

- Barra lateral izquierda fija con estas secciones:
  - Logo “ComuGest IA”.
  - Inicio (Home).
  - Incidencias.
  - Avisos.
  - Reservas.
  - Secretario IA (chat).
  - Ajustes (solo admin/presidente).
- Barra superior con:
  - Selector de comunidad (dropdown con nombre de la comunidad).
  - Nombre del usuario y rol (Vecino, Admin, Presidente).
  - Iconos: notificaciones y ayuda.

3. Pantallas para VECINO

Diseña estas pantallas en versión desktop y que se vean bien también en móvil:

3.1. Pantalla de Login / Acceso

- Formulario sencillo con:
  - Campo email o teléfono.
  - Campo “Código de comunidad”.
  - Botón “Entrar” (pensado para magic link o código OTP).
- Diseño minimalista, con un pequeño bloque de texto explicando la app.

3.2. Home del vecino

- Layout tipo dashboard, con:
  - Arriba: bienvenida (“Hola, [Nombre]”) y nombre de la comunidad.
  - Columna principal:
    - Sección “Incidencias recientes” (lista de 3–5 incidencias con estado).
    - Sección “Últimos avisos” (2 avisos importantes con fecha).
  - Columna lateral derecha:
    - Tarjeta “Próximas reservas” (si tiene reservas activas).
- Botón destacado “Nueva incidencia” siempre visible.
- Botón flotante en esquina inferior derecha para abrir el chat del “Secretario IA”.

3.3. Lista de incidencias (vecino)

- Vista tipo tabla o tarjetas, con columnas/campos:
  - Título.
  - Categoría (ascensor, garaje, limpieza, ruido, otros…).
  - Prioridad (baja, media, alta) con badges de color.
  - Estado (Abierta, En curso, Resuelta) con badges.

- Fecha de creación.
- Filtros en la parte superior: por estado y por categoría.
- Botón “Nueva incidencia” en la parte superior derecha.

3.4. Detalle de incidencia (vecino)

- Mostrar: título, descripción, categoría, prioridad, estado, fechas.
- Carrusel / grid de fotos y vídeos adjuntos.
- Timeline o lista de comentarios (tipo chat) entre vecino y admin.
- Si la incidencia está Resuelta, mostrar un bloque “Resumen de solución” destacado.

3.5. Lista de avisos

- Lista limpia de avisos ordenados por fecha, cada uno con:
  - Título, etiqueta de tipo (Obras, Junta, Limpieza, General).
  - Fecha de inicio y fin de vigencia.
- Al hacer clic, detalle del aviso en una tarjeta grande con el texto completo.

3.6. Reservas de espacios (vecino)

- Sección para elegir espacio (pista pádel, sala reuniones, etc.) con tarjetas.
- Tras elegir espacio, vista de calendario simple:
  - Vista semanal con bloques horarios.
  - Slots libres en color suave, slots ocupados en gris.
- Formulario de nueva reserva: fecha, hora inicio y fin, notas.
- Tarjeta lateral con “Reglas de uso del espacio” (texto breve).

3.7. Chat del “Secretario IA”

- Diseño tipo chat moderno:
  - Burbujas de usuario a la derecha.
  - Respuestas del secretario IA a la izquierda con un avatar de asistente.
- Campo de texto con botón de enviar y un pequeño texto de ayuda (“Puedes preguntar sobre normas, pagos, avisos, etc.”).

4. Pantallas para ADMIN / PRESIDENTE

Estas pantallas comparten el mismo layout general, pero con más controles:

4.1. Panel de incidencias (admin)

- Tabla más completa que la del vecino, con columnas:
  - Comunidad (si el admin gestiona varias).
  - Vecino (nombre y piso).
  - Categoría.
  - Prioridad.

    - Estado.
    - Fecha de creación.
- Filtros por comunidad, estado, categoría y prioridad.
- Al seleccionar una incidencia, panel lateral o pantalla de detalle con:
    - Datos completos.
    - Comentarios.
    - Selector de estado (dropdown).
    - Botones “Asignar a proveedor” y “Marcar como resuelta”.

4.2. Gestión de avisos (admin)

- Lista de avisos con botón “Nuevo aviso”.
- Pantalla de creación/edición de aviso:
    - Campo título.
    - Textarea para borrador de mensaje.
    - Selector de tipo de aviso.
    - Selector de fechas.
    - Botón “Pedir ayuda al Secretario IA” que genera un mensaje formal (de momento como botón secundario visual, no necesitas lógica aquí, solo el diseño).

4.3. Gestión de reservas (admin)

- Vista de calendario por espacio, con filtro por tipo de espacio.
- Tarjetas o tabla con reservas: vecino, fecha/hora, estado.
- Posibilidad visual de cancelar una reserva.

4.4. Ajustes de comunidad

- Pestañas dentro de Ajustes:
    - “Datos generales” (nombre comunidad, dirección, logo).
    - “Usuarios y roles” (lista de vecinos, con rol y piso).
    - “Normas y documentos” (lista editable de normas que la IA usará como contexto).
    - “Plan y facturación” (solo diseño: indicar si es Plan Básico o Pro, estado de suscripción).

5. Estados, feedback y componentes reutilizables

- Diseña estados vacíos para:
    - Sin incidencias.
    - Sin avisos.
    - Sin reservas.
- Incluye:
    - Un sistema de tarjetas reutilizables para incidencias, avisos y reservas.
    - Un sistema de badges de estado y prioridad.

- Modales de confirmación para acciones críticas (cerrar incidencia, cancelar reserva).

6. Responsiveness

- Diseña pensando en **mobile-first**, pero con versión desktop principal.
- En móvil:
  - Menú lateral convertido en bottom nav con iconos (Inicio, Incidencias, Avisos, Reservas, IA).
  - Tablas simplificadas a listas con la información más importante.

7. Entregable

- Genera todas las pantallas interconectadas como un **flujo completo de app** (no solo una vista), de forma que luego pueda exportarlas y usarlas en un proyecto con Antigravity.
- Prioriza: claridad de información, estructura de dashboard SaaS y coherencia de estilo entre todas las pantallas.

Quiero que construyas una aplicación SaaS llamada **"ComuGest IA"** para gestionar comunidades de vecinos y garajes. La app debe estar lista para producción (no solo prototipo) y usar:

- Frontend web (SPA) responsive.
- **Supabase** como backend (Postgres, Auth, Storage).
- **OpenRouter** como API para los modelos de IA (chat y generación de texto).

0. Objetivo general

La app debe permitir a presidentes, administradores y vecinos:

- Crear y gestionar **incidencias** (averías, limpieza, ruido…).
- Publicar y ver **avisos/comunicados**.
- Gestionar **reservas** de espacios comunes.
- Usar un **chat de "Secretario IA"** que:
  - Responda preguntas frecuentes sobre normas, pagos y avisos.
  - Ayude a redactar comunicados formales a partir de un borrador.
  - Resuma actas o textos largos en puntos clave.

Quiero que crees: frontend, backend, modelo de datos en Supabase, integración con OpenRouter y toda la lógica básica de negocio.

---

1. Stack técnico

- Frontend:
  - Web app con un framework moderno (por ejemplo React/Next.js generado por ti) pero mantenible.
  - Estilo tipo SaaS moderno (puedes guiarte por el diseño que importaremos desde Stitch).
- Backend:
  - Supabase para:
    - Base de datos Postgres.
    - Auth (email + magic link o email + contraseña).
    - Row Level Security (RLS) para separar comunidades.
    - Storage para fotos/vídeos de incidencias.
- IA:
  - OpenRouter para llamadas a modelos de chat/completions.
  - Usa la API HTTP estándar de OpenRouter (`https://openrouter.ai/api/v1/chat/completions`) con API key en variable de entorno.

Por favor:

- Configura todo el acceso a Supabase mediante el **Supabase MCP** (si está disponible) y genera el SQL o las migraciones necesarias.
- Crea un pequeño **módulo de cliente** para llamar a OpenRouter desde el backend (un wrapper simple).

---

2. Modelos de datos en Supabase

Crea estas tablas (puedes ajustar nombres, pero mantén conceptos):

1. `communities`
   - id (uuid, PK).
   - name (text).
   - address (text).
   - code (text único, para invitar usuarios).
   - plan (text: 'basic' | 'pro').
   - subscription_status (text: 'active' | 'trial' | 'paused' | 'cancelled').
   - created_at (timestamp).
2. `profiles`
   - id (uuid, PK, referencia a auth.users.id).
   - community_id (uuid, FK communities).
   - full_name (text).
   - email (text).
   - phone (text).
   - role (text: 'neighbor' | 'admin' | 'president').
   - unit (text, piso/puerta).
   - created_at.

3. `incidents`

- id (uuid, PK).
- community_id (uuid, FK).
- created_by (uuid, FK profiles).
- title (text).
- description (text).
- category (text: 'elevator' | 'garage' | 'cleaning' | 'noise' | 'other').
- priority (text: 'low' | 'medium' | 'high').
- status (text: 'open' | 'in_progress' | 'closed').
- ai_summary (text, opcional, resumen generado por IA).
- created_at, updated_at.

4. `incident_attachments`

- id (uuid, PK).
- incident_id (uuid, FK).
- file_url (text).
- file_type (text: 'image' | 'video' | 'other').
- created_at.

5. `incident_comments`

- id (uuid, PK).
- incident_id (uuid, FK).
- author_id (uuid, FK profiles).
- message (text).
- created_at.

6. `notices`

- id (uuid, PK).
- community_id (uuid, FK).
- created_by (uuid, FK profiles).
- title (text).
- body (text, versión larga).
- short_body (text, versión corta para WhatsApp/email).
- type (text: 'general' | 'meeting' | 'maintenance' | 'cleaning' | 'works').
- start_date (date).
- end_date (date).
- created_at.

7. `spaces`

- id (uuid, PK).
- community_id (uuid, FK).
- name (text).
- description (text).
- rules (text).
- created_at.

8. `reservations`

- id (uuid, PK).
- community_id (uuid, FK).
- space_id (uuid, FK).
- user_id (uuid, FK profiles).
- start_time (timestamptz).
- end_time (timestamptz).
- status (text: 'confirmed' | 'cancelled').

- created_at.

9. `faqs` / `docs` (elige uno)
    - id (uuid, PK).
    - community_id (uuid, FK).
    - title (text).
    - body (text).
    - type (text: 'rules' | 'payment' | 'other').
    - created_at.
10. `ai_sessions` (opcional para histórico de chat)
    - id (uuid, PK).
    - community_id (uuid, FK).
    - user_id (uuid, FK profiles).
    - messages (jsonb) con historial condensado.
    - created_at, updated_at.

Configura **RLS** para que:

- Cada usuario solo pueda acceder a datos de su `community_id`.
- Los roles `admin` y `president` vean todas las incidencias/avisos/reservas de su comunidad.
- Los `neighbor` vean solo sus incidencias (y, si lo defino, las públicas de la comunidad).

---

3. Autenticación, roles y comunidades

Implementa:

- Auth Supabase: login por email (puede ser magic link u otra forma estándar).
- Después del login, el usuario debe tener un registro en `profiles` con su comunidad y rol.
- Flujo de onboarding básico:
    - Un `admin`/`president` crea una comunidad (desde un panel admin).
    - Genera un `code` único de comunidad.
    - Los vecinos se registran usando ese código para asociarse a la comunidad.

---

4. Funcionalidad de negocio

4.1 Incidencias

- Vecino puede:
  - Crear incidencia (título, descripción, adjuntar fotos/vídeos).
  - Ver lista de incidencias (propias y, opcional, de la comunidad).
  - Comentar en sus incidencias.
- Admin/Presidente puede:
  - Ver todas las incidencias de la comunidad.
  - Cambiar estado y prioridad.
  - Añadir comentarios internos (mismo sistema de comentarios está bien).
- Lógica IA (backend):
  - Al crear incidencia, llamar a OpenRouter para:
    - Clasificar categoría y prioridad a partir de título + descripción.
    - Generar un `ai_summary` corto.

4.2 Avisos

- Admin/Presidente puede:
  - Crear, editar y cerrar avisos.
  - Escribir un borrador corto y pedir al "secretario IA" que genere:
    - Un texto formal (`body`).
    - Un texto corto (`short_body`) para WhatsApp/email.
- Vecino puede:
  - Ver lista de avisos activos.
  - Abrir el detalle de un aviso.

4.3 Reservas de espacios

- Vecino:
  - Ver espacios disponibles de su comunidad.
  - Ver calendario simple de reservas por espacio (slots ocupados/libres).
  - Crear reserva si hay hueco.
- Admin/Presidente:
  - Ver todas las reservas.
  - Cancelar o cambiar estado.
- Valida en backend que:
  - No haya solapamiento de reservas para el mismo `space_id`.
  - Se puedan definir más adelante límites por usuario (déjalo preparado).

4.4 "Secretario IA" (chat y utilidades)

Crea una API interna de IA (por ejemplo `/api/ai/assistant`) que:

- Use OpenRouter para responder en español, tono educado y claro.
- Cuando un usuario pregunte algo en el chat, haz lo siguiente:

1. Recupera datos relevantes de Supabase:
    - Normas (`faqs/docs` de la comunidad).
    - Últimos `notices` activos.
2. Construye un prompt de sistema incluyendo:
    - Descripción de la app y rol del asistente ("secretario de comunidad").
    - Normas y avisos que puedan ser relevantes.
3. Envía la pregunta del usuario como mensaje de usuario a OpenRouter.
4. Devuelve la respuesta y guárdala si lo crees útil en `ai_sessions`.

- Endpoint adicional para generar textos:
    - `POST /api/ai/generate-notice`: recibe un borrador y devuelve `body` + `short_body`.
    - `POST /api/ai/summarize-doc`: recibe un texto largo (acta de reunión) y devuelve un resumen en puntos.

Asegúrate de que el módulo cliente de OpenRouter:

- Use variable de entorno `OPENROUTER_API_KEY`.
- Permita cambiar el modelo fácilmente (por defecto usar `openrouter/auto` o un modelo fiable de chat).

---

5. Frontend: páginas y componentes

Usa las pantallas ya definidas en el diseño (Stitch) y conectarlas al backend:

- Páginas para vecinos:
    - Login.
    - Home.
    - Lista de incidencias.
    - Detalle incidencia.
    - Lista de avisos.
    - Detalle aviso.
    - Reservas (lista de espacios + calendario + formulario).
    - Chat del secretario IA.
- Páginas para admin/presidente:
    - Panel de incidencias (lista + detalle).
    - Gestión de avisos (lista + creación/edición).
    - Gestión de reservas.
    - Ajustes de comunidad (datos, usuarios, normas, plan).

Componentes reutilizables:

- `IncidentCard`, `NoticeCard`, `ReservationCard`.

- `StatusBadge` para estados y prioridad.
- `ConfirmModal` para acciones críticas.
- `ChatWindow` para el secretario IA.

---

6. Planes y limitaciones (Básico vs Pro)

Dentro de la lógica, prepara un sencillo sistema de planes basado en campos de la tabla `communities`:

- Plan Básico (`plan = 'basic'`):
  - Acceso a incidencias y avisos.
  - Sin reservas.
  - Sin funciones IA avanzadas (opcionalmente limitar IA a FAQ).
- Plan Pro (`plan = 'pro'`):
  - Todo lo anterior + reservas + IA completa (chat + generación de comunicados + resúmenes).

Por ahora, no hace falta integrar Stripe:

- Implementa solo la lógica condicional en el frontend y backend para mostrar/ocultar features según el `plan`.
- Deja documentado y preparado un punto futuro de integración con Stripe.

---

7. Calidad, documentación y despliegue

- Asegúrate de que el código se pueda ejecutar localmente con un archivo `.env.example` que incluya:
  - Variables de Supabase (URL, anon key, service key si se necesita en backend).
  - `OPENROUTER_API_KEY`.
- Añade instrucciones en un README: cómo levantar el frontend, cómo conectar Supabase y cómo configurar las tablas.
- Diseña la app para poder desplegarse fácilmente en un proveedor típico de hosting de apps modernas (por ejemplo Vercel + Supabase).

Por favor, construye todo este proyecto de forma iterativa, comprobando errores y corrigiéndolos automáticamente si es posible, hasta dejar una versión estable y funcional lista para que yo la pruebe con comunidades reales.

Hay en el mercado cosas parecidas?

Sí, hay varias apps y softwares muy parecidos en funcionalidades “clásicas” (incidencias, avisos, reservas), pero casi ninguno está centrado en un **secretario IA** inteligente como núcleo del producto.